

WEEK 7

STUDENT COURSE MANAGEMENT SYSTEM

Using Node.js and Express.js

Aim

To develop a Student Course Management System using Node.js and Express.js that manages student information through HTTP GET requests, Express routes, URL parameters, query parameters, built-in Node.js modules, and a JSON file for storing student data.

Description

The application is a basic server-side web application. It uses Express.js to create the server and define routes. The fs module reads student information from students.json, while the os module provides system information and the dns module performs a DNS lookup for google.com. The application supports retrieving all students, retrieving a student by ID, searching students by course, viewing system information, and performing a DNS lookup.

Routes Used

Method	URL / Route	Purpose
GET	http://localhost:3000/students	Display all students
GET	http://localhost:3000/students/:id	Display a student using ID
GET	http://localhost:3000/search?course=Node.js	Search students by course
GET	http://localhost:3000/system	Display system information
GET	http://localhost:3000/dns	Perform DNS lookup

Program Code – server.js

The following code is the submitted server.js program.

```
001 const express = require("express");
002 const fs = require("fs");
003 const os = require("os");
004 const dns = require("dns");
005
006 const app = express();
007 const PORT = 3000;
008
009 function getStudents() {
010     const data = fs.readFileSync("students.json", "utf-8");
011     return JSON.parse(data);
012 }
013
014 // Home route
015 app.get("/", (req, res) => {
016     res.send("Student Course Management System");
017 });
018
019
020 app.get("/students", (req, res) => {
021     const students = getStudents();
022     res.json(students);
023 });
```

```

024
025
026 app.get("/students/:id", (req, res) => {
027     const students = getStudents();
028
029     const id = parseInt(req.params.id);
030
031     const student = students.find(s => s.id === id);
032
033     if (student) {
034         res.json(student);
035     } else {
036         res.status(404).send("Student not found");
037     }
038 });
039
040
041 app.get("/search", (req, res) => {
042     const students = getStudents();
043
044     const course = req.query.course;
045
046     const result = students.filter(
047         student => student.course.toLowerCase() === course.toLowerCase()
048     );
049
050     res.json(result);
051 });
052
053 app.get("/system", (req, res) => {
054     res.json({
055         platform: os.platform(),
056         architecture: os.arch(),
057         hostname: os.hostname(),
058         cpus: os.cpus().length,
059         memory: os.totalmem()
060     });
061 });
062
063
064 app.get("/dns", (req, res) => {
065     dns.lookup("google.com", (err, address, family) => {
066
067         if (err) {
068             return res.status(500).send("DNS lookup failed");
069         }
070
071         res.json({
072             hostname: "google.com",
073             address: address,
074             family: family
075         });
076     });
077 });
078
079
080
081
082 app.listen(PORT, () => {
083     console.log(`Server running at http://localhost:${PORT}`);
084 });

```

Data File – students.json

The student data used by the application is stored in the JSON file below.

```
001  [  
002    {  
003      "id": 1,  
004      "name": "Vani",  
005      "course": "Node.js",  
006      "year": 3  
007    },  
008    {  
009      "id": 2,  
010      "name": "Anu",  
011      "course": "Python",  
012      "year": 2  
013    },  
014    {  
015      "id": 3,  
016      "name": "Rao",  
017      "course": "Node.js",  
018      "year": 3  
019    },  
020    {  
021      "id": 3,  
022      "name": "Reeshma",  
023      "course": "Node.js",  
024      "year": 3  
025    }  
026  ]
```

Output

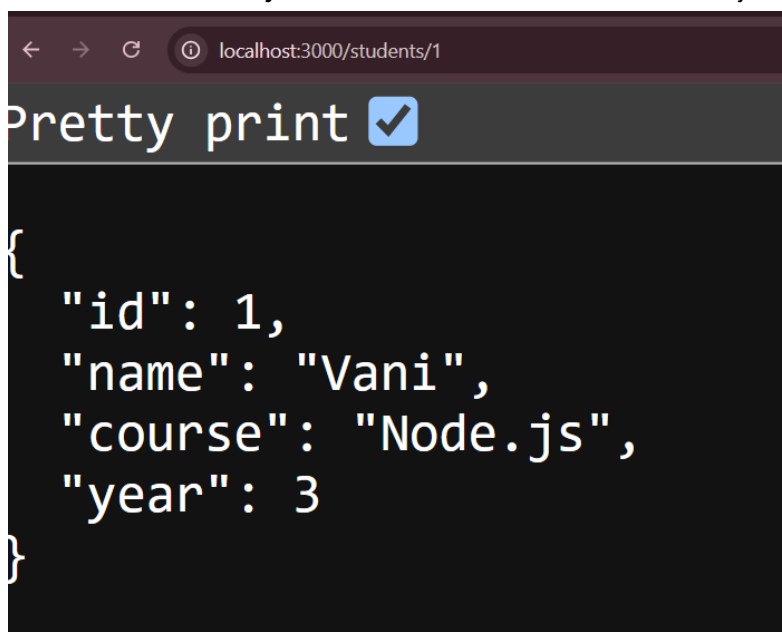
The application should be started using Node.js. The submitted program listens on port 3000 and prints a server-running message when the server starts. The routes return JSON data or text according to the requested operation. The `/students` route reads `students.json` and returns all student records, while the `/students/:id` route searches for a matching numeric ID

Sample Route Output

`/students` – student data returned from the JSON file:

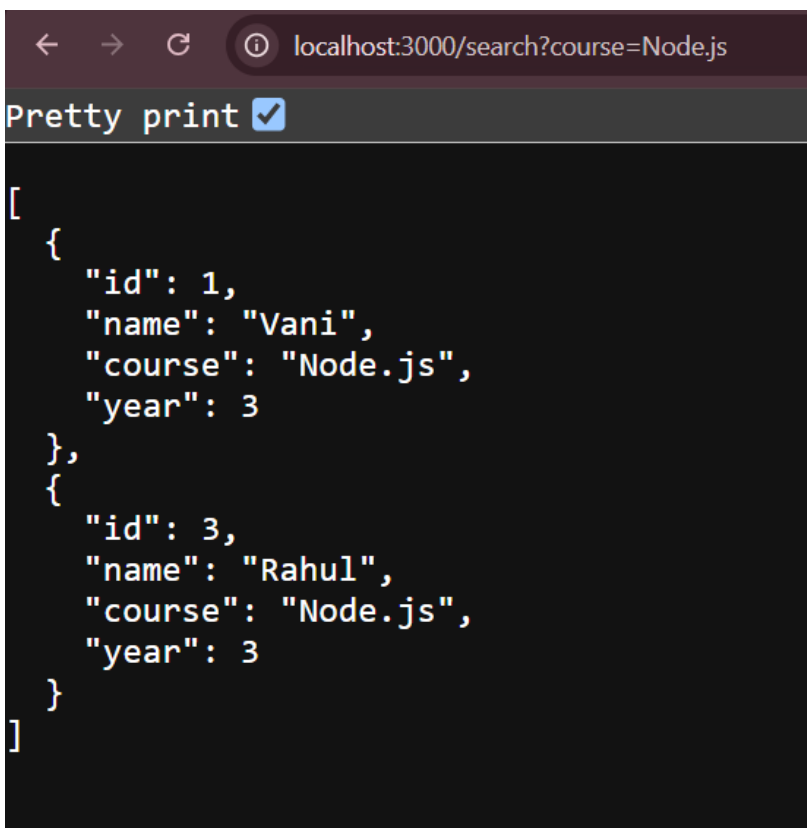
```
[
  {
    "id": 1,
    "name": "Vani",
    "course": "Node.js",
    "year": 3
  },
  {
    "id": 2,
    "name": "Anu",
    "course": "Python",
    "year": 2
  },
  {
    "id": 3,
    "name": "Rao",
    "course": "Node.js",
    "year": 3
  },
  {
    "id": 3,
    "name": "Reeshma",
    "course": "Node.js",
    "year": 3
  }
]
```

`/search?course=Node.js` – the submitted data contains three Node.js records (Vani, Rao and Reeshma).





Student Course Management System



Result

Thus, the Student Course Management System was developed using Node.js and Express.js. The application defines routes for student retrieval, ID-based lookup, course search, system information, and DNS lookup, while student information is stored in a JSON file. The submitted server code also starts the application on port 3000.